01

Como a LLM processa informação

Você digita um prompt, espera alguns segundos e recebe uma resposta que parece escrita por alguém que entendeu o pedido. Só que não houve leitura nem entendimento nesse caminho — houve matemática. Este material percorre as quatro etapas que acontecem entre o seu texto e a resposta, porque é nelas que mora a explicação para quase tudo que a IA faz de certo e de errado.

A IDEIA CENTRAL DESTE MATERIAL

A LLM não lê o seu texto. Ela converte texto em números, mede distâncias num mapa de significados e prevê a próxima palavra mais provável — uma de cada vez. Quem entende esse caminho entende por que contexto muda tudo.

1 Tokenização: o texto vira peças

Nada entra no modelo como palavra ou frase. O primeiro passo é sempre desmontar o texto em **tokens** — fragmentos que o modelo aprendeu a tratar como unidades durante o treinamento. Às vezes um token é uma palavra inteira; muitas vezes é um pedaço dela:

```
"the"            → ["the"]                  1 token
"API"            → ["API"]                  1 token
"autenticação"   → ["autent", "icação"]     2 tokens
"gerenciamento"  → ["ger", "enci", "amento"] 3 tokens
```

Repare no padrão: palavras frequentes em inglês costumam virar um único token; palavras técnicas, compostas ou em português tendem a ser quebradas em mais pedaços. A régua prática é **1 token ≈ 4 caracteres** em inglês — e um pouco mais custoso em português. Ou seja: o mesmo conteúdo, escrito em português, consome mais tokens.

Isso importa por um motivo direto: **tudo no mundo das LLMs é medido em tokens** — o limite de contexto, a cobrança da API, o tamanho da resposta. Não em palavras, não em caracteres, não em tempo.

> A tokenização é **determinística**: o mesmo texto, com o mesmo tokenizador, gera sempre exatamente os mesmos tokens. É a porta de entrada de todo o resto.

2 Embeddings: cada peça vira um ponto no mapa

Depois de desmontado, cada token é convertido em um **vetor** — uma longa lista de números que funciona como uma coordenada num espaço matemático de centenas ou milhares de dimensões. Pense num mapa de cidade absurdamente detalhado: cada token mora em algum endereço desse mapa.

A propriedade que faz tudo funcionar é uma só: **significados parecidos moram perto**. No mapa, formam-se "bairros" de significado:

BAIRRO	MORADORES PRÓXIMOS ENTRE SI
Saúde	médico · hospital · enfermeira · diagnóstico
Automotivo	carro · motor · freio · combustível
Financeiro	banco · investimento · juros · crédito

Aqui vale desfazer uma ilusão: o modelo não "sabe" que médico e hospital têm relação, do jeito que você sabe. O que aconteceu é estatística — esses tokens apareceram juntos, em contextos parecidos, bilhões de vezes durante o treinamento, e essa coocorrência virou proximidade no mapa. **O significado emergiu da geometria**, não de compreensão.

3 O contexto move os pontos

Se cada palavra tivesse um endereço fixo no mapa, a coisa toda quebraria na primeira ambiguidade. A palavra "banco" resolve isso: o vetor de um token **não é fixo** — ele é recalculado a cada uso, com base nas palavras ao redor.

```
"fui ao banco sacar dinheiro"   → vizinho de: dinheiro · agência · conta
"sentei no banco da praça"      → vizinho de: praça · sentar · descanso
"banco de dados da aplicação"   → vizinho de: SQL · tabela · query
```

Essa é a consequência mais prática de todo este material: **o contexto que você escreve no prompt não é "informação a mais" — ele literalmente muda os números que o modelo calcula**. Um prompt vago produz vetores genéricos, que ativam os padrões mais batidos do treinamento. Um prompt específico empurra os vetores para regiões precisas do mapa — e a resposta vem de lá.

> Quando alguém diz que "a IA respondeu genérico", quase sempre a causa é esta: o prompt mandou o modelo para o **centro do mapa**, onde moram as respostas médias para perguntas médias.

4 Atenção: como o modelo liga as palavras

Falta um mecanismo para o contexto conseguir mover os pontos: a **atenção**. Para cada token, o modelo calcula o quanto cada outro token da sequência importa para definir o seu significado — todos os pares, ao mesmo tempo.

Você faz isso sem perceber. Ao ler *"o banco estava cheio de gente"*, seu cérebro conecta "banco" com "gente" e resolve a ambiguidade na hora: é banco de praça. A atenção faz exatamente essa conta:

```
"o banco estava cheio de gente"
      ↑                        ↑
      └──────── atenção ───────┘
resultado: banco de praça, não banco financeiro
```

Como os pesos são calculados para todos os pares simultaneamente — e não palavra por palavra, em sequência — o modelo consegue conectar coisas distantes no texto. É isso que permite manter coerência em documentos longos, e é por isso que modelos com janelas de contexto maiores conseguem "segurar" conversas mais extensas.

5 A resposta nasce um token por vez

Com os vetores calculados e as relações mapeadas, chega a etapa final — e ela é menos glamourosa do que parece: o modelo calcula, para **todo o vocabulário**, a probabilidade de cada token ser o próximo. E escolhe um. Depois recomeça a conta, agora incluindo o token que acabou de gerar. Uma resposta de 500 tokens são 500 rodadas dessa previsão.

Um parâmetro chamado **temperatura** controla o quanto essa escolha é conservadora ou ousada:

TEMPERATURA	COMPORTAMENTO	USE PARA
Baixa (perto de 0)	Escolhe quase sempre o token mais provável. Respostas estáveis e repetíveis.	Código, tarefas técnicas, extração de dados
Alta (perto de 1+)	Dá chance a tokens menos prováveis. Respostas variadas e criativas.	Escrita criativa, brainstorm, variações

6 O limite: o modelo não sabe que não sabe

Agora a consequência mais séria da arquitetura. O modelo **sempre** executa a mesma operação — prever o próximo token mais provável — tanto quando tem a informação quanto quando não tem. Não existe um circuito interno de "opa, isso eu não sei".

```
✓ "qual é a capital da França?"
  → padrão fortíssimo no treinamento → "Paris"

✗ "qual método de autenticação este projeto usa?"
  → o projeto não estava no treinamento
  → o modelo interpola padrões parecidos
  → e responde algo plausível, com a mesma confiança
```

Essa é a origem das **alucinações** — e vale registrar: não é um bug que uma próxima versão vai corrigir. É consequência direta de como a predição funciona. Aliás, modelos maiores alucinam de forma *mais convincente*, porque conhecem mais padrões para interpolar.

> **A única defesa é contexto.** Se o fato específico não estiver na janela de contexto — via prompt, via arquivos do projeto, via RAG — o modelo preenche a lacuna com o que é estatisticamente provável. E o estatisticamente provável é o centro do mapa, não o seu caso.

7 O que isso muda no seu dia a dia

Cada comportamento estranho que você já viu numa LLM tem explicação direta nas etapas acima:

COMPORTAMENTO QUE VOCÊ OBSERVA	O QUE ESTÁ ACONTECENDO POR BAIXO
O mesmo prompt dá respostas diferentes em sessões diferentes	O contexto acumulado da conversa entra no cálculo dos vetores — sessões diferentes, vetores diferentes.
Prompts detalhados funcionam melhor que prompts curtos	Cada detalhe empurra os vetores para regiões mais específicas do mapa, ativando padrões mais relevantes.
Em conversas longas, a IA “esquece” instruções do início	A atenção cobre todos os tokens, mas os mais distantes tendem a pesar menos. Repetir instruções críticas renova o peso delas.
Trocar por um modelo maior não acabou com as alucinações	Mais parâmetros = mais padrões para interpolar. O modelo continua sem saber o que não sabe.
Um CLAUDE.md/AGENTS.md bem escrito melhora tudo consistentemente	Ele injeta contexto do projeto antes de qualquer prompt — os vetores já começam na região certa do mapa.

▸ **COLA PARA FIXAR**

- **Token** é a unidade de tudo: contexto, cobrança e limite se medem em tokens, e português custa mais que inglês.
- **Embedding** é o endereço do token num mapa de significados — perto no mapa = parecido no sentido.
- O endereço **não é fixo**: as palavras ao redor recalculam o vetor. Contexto não é acessório — é parte da conta.
- A **atenção** conecta todos os pares de tokens de uma vez e resolve ambiguidades como um leitor humano faria.
- A resposta é gerada **um token por vez**, sempre o mais provável — nunca "o verdadeiro".
- O modelo **não sabe que não sabe**: sem contexto, ele inventa com confiança. Alucinação é arquitetura, não defeito.